Front matter

Department of Electronic Systems

Examination paper forTFE4141 Design of Digital Systems 1

Examination date: December 20, 2022

SOLUTION EXAMPLE

Examination time (from-to): 15:00 – 19:00

Permitted examination support material: A / All printed and hand-written support material is allowed. All calculators are allowed.

Academic contact during examination: Per Gunnar Kjeldsberg
Phone: 934 59 550

OTHER INFORMATION

Get an overview of the question set before you start answering the questions.

Read the questions carefully and make your own assumptions. If a question is unclear/vague, make your own assumptions and specify them in your answer. Some design tools (such as Vivado) may behave differently than what is specified in the VHDL standard. Answers to the questions in this exam should reflect the standard as it is presented in lectures and textbook. Only contact academic contact in case of errors or insufficiencies in the question set. Address an invigilator if you wish to contact the academic contact. Write down the question in advance.

InsperaScan: For questions **13** and **14** it is possible to submit the entire/some parts of the answer as handwritten sheets. At the bottom of the question you will find a seven-digit code. Fill in this code in the top left corner of the sheets you wish to submit. We recommend that you do this during the exam. If you require access to the codes after the examination time ends, click “Show submission”.

Weighting: The total number of achievable points is 100. The maximum number for each question is given on each question cover page.

Notifications: If there is a need to send a message to the candidates during the exam (e.g. if there is an error in the question set), this will be done by sending a notification in Inspera. A dialogue box will appear. You can re-read the notification by clicking the bell icon in the top right-hand corner of the screen.

Withdrawing from the exam: If you become ill or wish to submit a blank test/withdraw from the exam for another reason, go to the menu in the top right-hand corner and click “Submit blank”. This cannot be undone, even if the test is still open.

Access to your answers: After the exam, you can find your answers in the archive in Inspera. Be aware that it may take a working day until any hand-written material is available in the archive.

1 Logic signals (Correct answer 2 points, wrong answer -1 point, no answer 0 points)

Given the code below, first some signal declarations, and then two logic statements further down in the code. Assume that A and B have been high for a long time. Then a 1 ns low pulse appears on A before it returns to high. Which of the alternatives below is correct with respect to the resulting values on signals C and D?

```
signal A, B, C, D : STD_LOGIC := '1';

...

C <= A and B after 2 ns;
D <= transport A xnor B after 2 ns;
```

Select one alternative:

- Both of the other alternatives are wrong.
- D stays high while C has a 1 ns low pulse.
- ☒ C stays high while D has a 1 ns low pulse.

Maximum marks: 2

Solution example:
The default is inertial delay, where pulses shorter then the assigned delay is suppressed. This is also how a real circuit would work due to load capacitances. When delay mechanism "transport" is specifically assigned, all pulses, no matter how short, will propagate, however.

2 Bit pattern (Correct answer 2 points, wrong answer -1 point, no answer 0 points)

What is the resulting bit pattern on DataOut of the following VHDL statement?

```
DataOut <= "10011100" sra 3;
```

Select one alternative:

- X "11110011"
- "00010011"
- "11100000"

Maximum marks: 2

Solution example:
Shift Right Arithmetic (SRA) shifts the bit pattern towards the right the indicated number of bits. The value '1' is added on the left side.

3 Type of circuit

Given the code below. What type of circuit does it model.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity QuestionCircuit is
    port (  clk : in std_logic;
            sdi : in std_logic;
            sdo : out std_logic);
end QuestionCircuit;

architecture rtl of QuestionCircuit is
    signal data_n: std_logic_vector(3 downto 0);
    signal data_r : std_logic_vector(3 downto 0);
begin

    process(clk)
    begin
        if(clk'event and clk='1') then
            data_r <= data_n;
        end if;
    end process;

    process(sdi, data_r)
    begin
        data_n <= data_r(2 downto 0) & sdi;
    end process;

sdo <= data_r(3);

end rtl;
```

Select one alternative:

- [X] A shift register
- [ ] A regular register
- [ ] A scan chain for use in production testing

Maximum marks: 2

Solution example:
Serial data is inserted on sdi and concatenated with the three currently least significant bits to give the new register value. The most significant bit is thus discarded, resulting in a shift register.

4 Subtype (Correct answer 2 points, wrong answer -1 point, no answer 0 points)

Given the definition of the type std_ulogic:

```
TYPE std_ulogic IS ( 'U', -- Uninitialized
                     'X', -- Forcing Unknown
                     ‘0’, -- Forcing 0
                     ‘1’, -- Forcing 1
                     ‘Z’, -- High Impedance
                     ‘W’, -- Weak Unknown
                     ‘L’, -- Weak 0
                     ‘H’, -- Weak 1
                     ‘-‘, -- Don't care );
```

Assume that you have defined the following subtype

```
SUBTYPE my_logic IS std_ulogic range 'X' to 'Z';
```

You then try to assign a sequence of the following values to a signal of type my_logic: '0', 'Z' 'L'. How many of these are you allowed to assign to the signal?

Select one alternative:

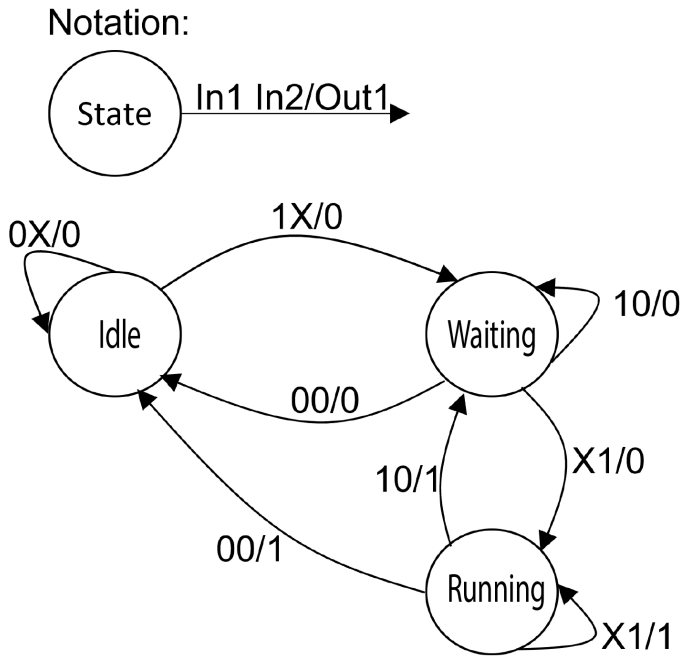
- ○ All three
- ○ Only one
- ⊗ Two

Maximum marks: 2

Solution example:
The subtype is defined for the value range between 'X' and 'Z', i.e., 'X', '0', '1', and 'Z'. Assignment of '0' and 'Z' is hence OK, while 'L' is illegal according to the VHDL standard.

5 State Machine (Correct answer 2 points, wrong answer -1 point, no answer 0 points)

Given the state diagram below and the three alternative sequences of input values on In1 and In2. Assume that In1 and In2 change value on the negative flank of a clock signal, while the state machine changes state on the positive flank of the same clock signal. Assume further that the state machine starts in state Idle. Which sequence of values will result in a single one clock period long positive pulse on Out1 (which otherwise is low)?



Sequence A		Sequence B		Sequence C	
In1	In2	In1	In2	In1	In2
0	0	1	0	0	1
0	1	1	0	1	1
1	1	1	1	0	1
1	0	1	1	1	0
0	0	0	0	1	0

Select one alternative:

- X Sequence C
- Sequence B
- Sequence A

Maximum marks: 2

Solution example:
Sequence A never enters state "Running" and consequently do not give out a positive pulse at all.
Sequence B stays for two clock periods in "Running", and consequently gives out a two clock periods long pulse.
Sequence C stays in "Running" for one clock period, as required.

6 States in state machine (Correct answer 2 points, wrong answer -1 point, no answer 0 points)

Given the FSM in the previous task. What is the minimum number of flip-flops in the synthesized circuit?

Select one alternative:

- 1
- X 2
- 3

Solution example:
Since we have three states, we need to be able to code three different state encodings. This requires to bit, and consequently two flip-flops to store the current state.

Maximum marks: 2

7 State machine coding (Correct answer 2 points, wrong answer -1 point, no answer 0 points)

Given the state machine in the two previous tasks and the VHDL code below. The code shows a part of the combinatorial process in the recommended format but with modified state names. Which state in the state diagram does QuestionState correspond to?

```
when QuestionState =>
    Out1 <= '0';
    if In1 = '0' and In2 = '0' then
        next_state <= OtherState1;
    elsif In1 = '1' and In2 = '0' then
        next_state <= QuestionState;
    else
        next_state <= OtherState2;
    end if;
```

Select one alternative:

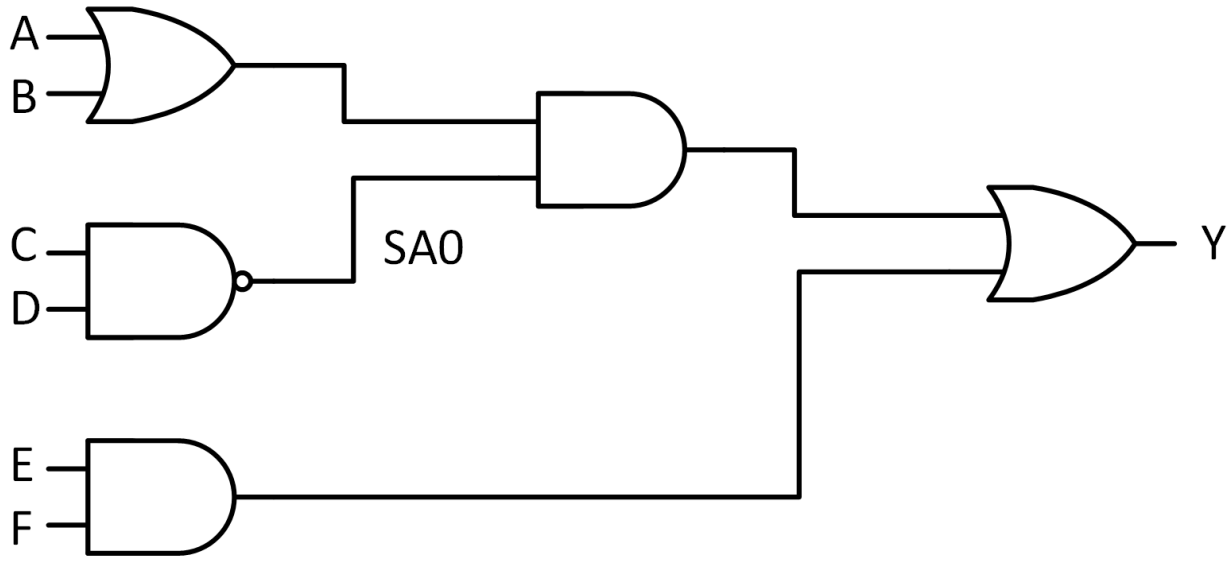
- X Waiting
- Idle
- Running

Solution example:
We see that Out1 is low, en consequently it must be Idle or Waiting. We also see that we stay in QuestionState when In1 = '1' and In2 = '0'. This is the case for Waiting.

Maximum marks: 2

8 Test patterns (Correct answer 2 points, wrong answer -1 point, no answer 0 points)

Given the circuit below. What pattern on the primary inputs can NOT be used to detect the output of the NAND gate stuck at 0?



Select one alternative:

X ABCDEF = 110111

ABCDEF = 100110

ABCDEF = 111001

Solution example:
A test pattern for the output of the NAND gate SA0 requires the signal to be controlled to '1'. This is the case if C or D (or both) are '0'. This is the case for all three alternatives. In addition, the fault has to be observable on the output Y. This requires A or B (or both) to be '1' so that the other input of the AND gate is '1'. This is again the case for all three alternatives. Finally, the other input of the last OR-gate needs to be '0'. This requires that E or F (or both) are '0'. This is not the case for the selected alternative.

Maximum marks: 2

9 True or false (Correct answer 2 points, wrong answer -1 point, no answer 0 points)

Which of the following statements is FALSE?

Select one alternative:

- X A signal of type std_logic can not have more than one driving sources.
- A delay-fault found during production testing may be caused by variations in doping in a circuit.
- In many digital systems, accessing the memory subsystem consumes a major part of the total power.

Solution example: std_logic has a built in resolution function. Consequently, multiple drivers are legal (even though not necessarily recommended). The other statements are true.

Maximum marks: 2

10 True and false (Correct answer 2 points, wrong answer -1 point, no answer 0 points)

Which one of the following statements is true:

Select one alternative:

- Generic constants are included in the port list of an entity.
- Generic constants are mainly used for debugging of code.
- X Generic constants can simplify reuse of entities between designs.

Maximum marks: 2

Solution example:
Generic constants are included in a separate generics list in an entity and are not used for debugging. Since they make it possible to, e.g., change the word width of signals internal to the entity, it makes reuse easier in situations where different width are needed.

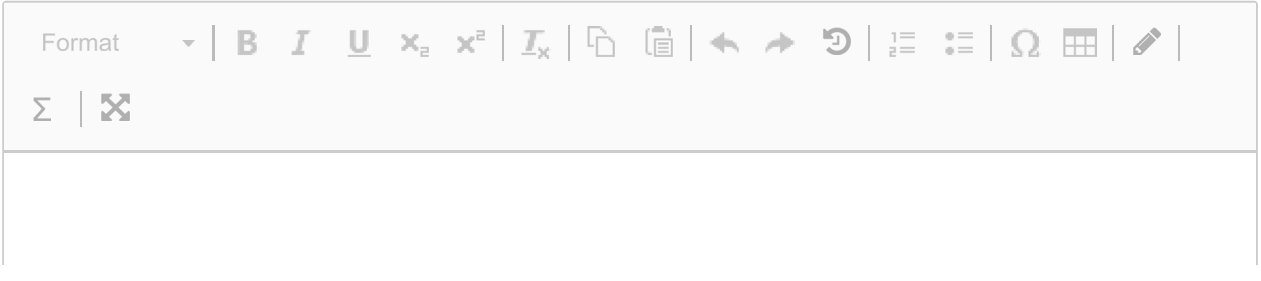
11 Inferred Latches (Maximum 12 points)

The code below will infer latches during synthesis. Answer the following questions in the text box below.

- Why is inferred latches in general a problem?
- Identify the parts of the code that results in latches and explain what the problem with the code is.
- Rewrite the part of the code that results in latches in such a way that the latches are avoided (you do not have to repeat the rest of the code).

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity InferredLatches is Port ( a, b, c, d : in std_logic_vector(15 downto 0);
                                 operation : in std_logic_vector( 1 downto 0);
                                 z : out std_logic_vector(15 downto 0);
                                 y : out std_logic_vector(15 downto 0) );
end InferredLatches;

architecture rtl of InferredLatches is
    constant A_ADD_B : std_logic_vector(1 downto 0):= "00";
    constant A_SUB_B : std_logic_vector(1 downto 0):= "01";
    constant C_ADD_D : std_logic_vector(1 downto 0):= "10";
    constant C_SUB_D : std_logic_vector(1 downto 0):= "11";
    signal result1 : signed(15 downto 0);
    signal result2 : signed(15 downto 0);

begin

    process(a,b,c,d, operation)
    begin
        case(operation) is
            when A_ADD_B =>
                result1 <= signed(a) + signed(b);
            when A_SUB_B =>
                result2 <= signed(a) - signed(b);
            when C_ADD_D =>
                result1 <= signed(c) + signed(d);
            when others => -- C_SUB_D
                result2 <= signed(c) - signed(d);
        end case;
     end process;

     y <= std_logic_vector(result1);
     z <= std_logic_vector(result2);

 end rtl;
```

Fill in your answer here



Solution example:
Inferred latches typically result from incorrect code that is intended to be combinatorial but ends up having to have some kind of memory. Thus the resulting outputs will depend not only on the current inputs, but also on some old state of these unintended latches.

The value stored in the latch may often be random and the resulting circuit will therefore not only produce a wrong result, it can also be impossible to predict what this wrong value will be. It will often be a timing dependent random value.

Even though the circuit may behave correctly during simulations, it might not work in a real implementation.

In the code, we assign a value to either result1 or result2 in each case statement. When the operation changes, the circuit hence has to remember the old value of the output that is currently not assigned.

Words: 0

Maximum marks: 12

To avoid latches we must make sure all outputs of the process are given a value for all input combinations. This can be done for example as indicated below.

```
process(a,b,c,d, operation)
begin
   case(operation) is
      when A_ADD_B =>
         result1 <= signed(a) + signed(b);
         result2 <= (others => '0');
      when A_SUB_B =>
         result1 <= (others => '0');
         result2 <= signed(a) - signed(b);
      when C_ADD_D =>
         result1 <= signed(c) + signed(d);
         result2 <= (others => '0');
      when others => -- C_SUB_D
         result1 <= (others => '0');
         result2 <= signed(c) - signed(d);
   end case;
end process;
```

12 Voltage and frequency scaling (Maximum 10 points)

Explain why frequency scaling can save power but not energy, while voltage and frequency scaling can save both power and energy.

Fill in your answer here

Format

Solution example:

The equations for dynamic power and energy is given as follows in the lectures:

$P = a\ C\ V^2\ f$

$E = N\ C\ V^2$

where a is the activity factor (ratio of 0 -> 1 transitions compared to the clock frequency), C is the load capacitance, V is the supply voltage, f is the clock frequency, and N is the number of transitions needed to do a certain task.

As we can see, E is independent of f, and can hence only be reduced by scaling V (or reducing C or N). P, on the other hand, depends both on V and f, so even if we only reduce f, P will be reduced.

If we see E as power multiplied with the time it takes to execute the task, the benefit of reducing f will be canceled by the fact that the task takes longer to execute.

If we reduce f we may also be able to reduce V. Reducing V makes the circuit slower, but this is ok since each poitive clock pulse is further apart (due to reduced f), and hence we can allow a longer critical path in the circuit. Reducing both V and f gives us a cubic reduction in dynamic power and a quadratic reduction in dynamic energy.

Words: 0

Maximum marks: 10

13 RSA cryptography (Maximum 14 points)

Based on your experience from the term project, and general understanding of the RSA algorithm, what technique(s) would you use to speed up encryption of large messages?

You do not have to go into implementation details but should include a simplified block diagram / micro architecture to assist your explanation. This can be drawn on a separate sheet of paper to be marked with the control code and handed in at the end of the exam.

Fill in your answer here

Format

Solution example:
See example next page.

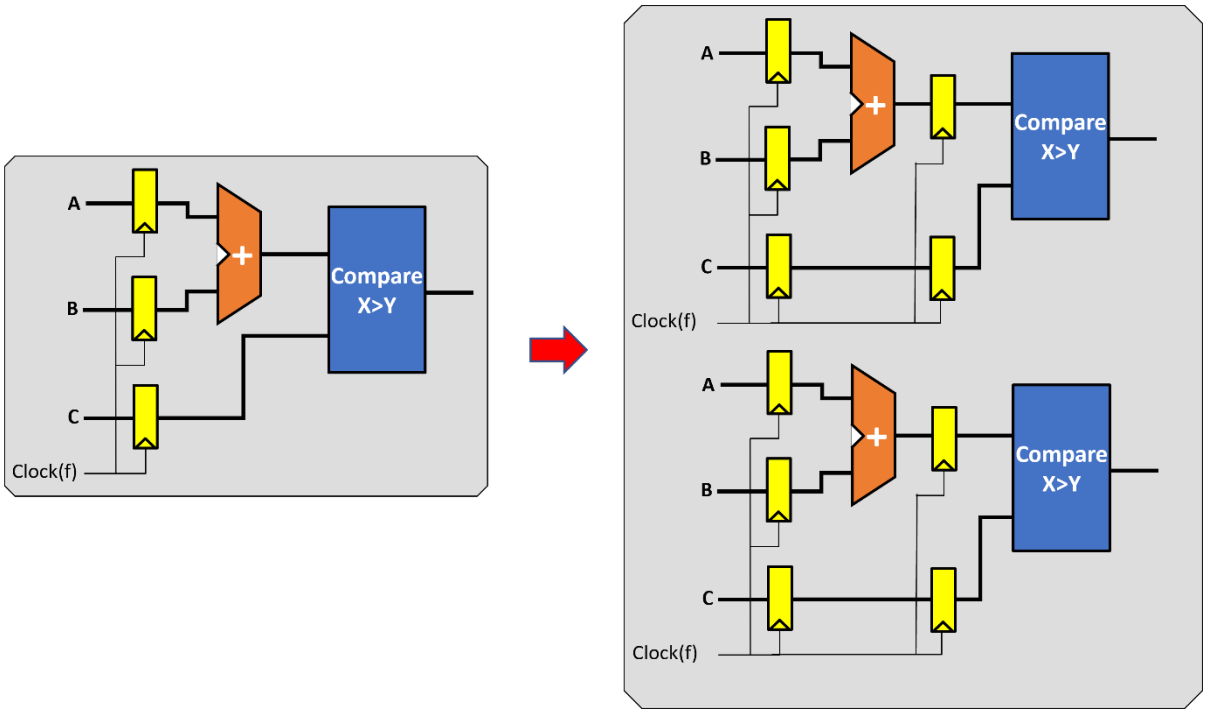
Words: 0

Maximum marks: 14

Solution example:

Increased encryption speed for large messages can be achieved either through very advanced and efficient implementation of the encryption algorithm itself, which may require considerable chip area, or through a simpler and less area costly implementation of the algorithm, which gives room for multiple cores that run in parallel.

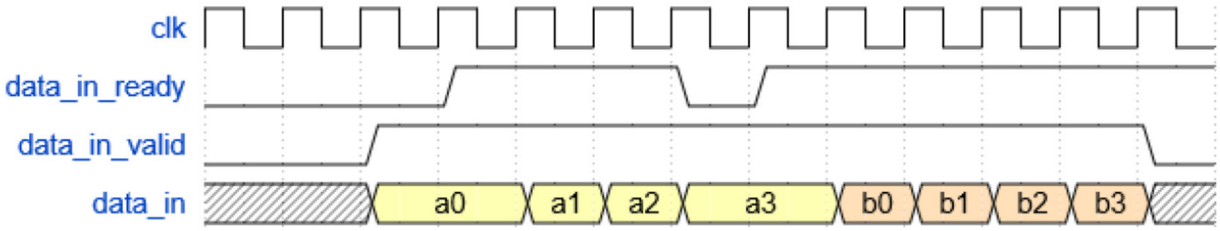
Different algorithms, e.g., Blakely versus Montgomery, have different area/performance/complexity trade-off points. After choosing the algorithm, there is typically also possibilities for internal parallelization, so that, e.g., several multiplications can be performed simultaneously. This will reduce the overall computation time, at the cost of a larger chip area. There is often also possible to insert pipeline registers in the design, so that you can start work on new data before the work on the previous data set(s) is completely finished. This can increase throughput even though the latency, in number of clock cycles, may increase. Due to shorter critical path between registers in the design, the clock frequency may also increase, partly compensating for the increased number of cycles. The figure below demonstrates the principles of parallelization and pipelining (though a figure related to the RSA algorithm would have been even better).



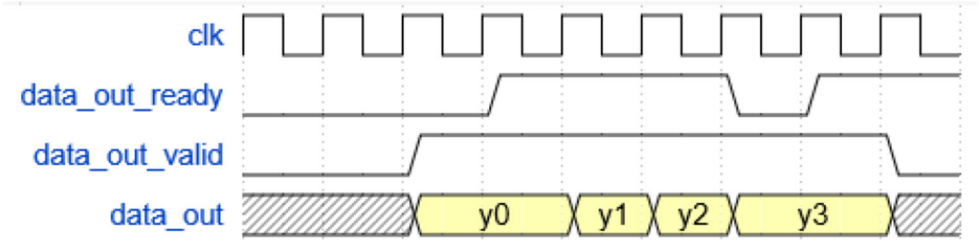
Depending on the area required for one RSA implementation, it may be possible to add one or more RSA cores in parallel. Each of them can then handle one 256 bit message chunk in parallel, thus dramatically speeding up the encryption/decryption of large multi-chunk messages. This can be demonstrated with a microarchitecture having an input unit that handles input handshaking and dispatches 256 bit messages to the parallel cores when they are ready for new work. At the output, an output unit receives messages from the cores and orders them correctly. It also performs output handshaking.

14 Design task (maximum 44 points)

You shall design a circuit that adds eight 16-bit signed numbers that arrive on a 64-bit wide data bus, with four numbers arriving in parallel each bus cycle. The final result shall be written to a separate result bus. A valid-ready protocol similar to the one used in the term project controls the communication on the two buses. See the following figure.



Input interface.



Output interface.

Solve the tasks below, using the the answer box as much as possible, but also by drawing and writing on separate sheets of paper when necessary. These should be marked with the control code and handed in at the end of the exam.

a) (max 10 points)

Draw a micro-architecture of the adder circuit for a design that calculates the result as quickly as possible. You can use as many adders and other required circuits as you want, but fewer is better as long as you do not increase the total time required to generate the final result. Indicate specifically what bit-width you need to have on the signals connecting modules in your design.

b) (max 8 points)

Consider a situation where the entity receiving the addition results is only ready to receive new data every five clock cycles. The entity giving you input data is able to give you four new numbers every clock cycle. Draw a timing diagram that shows how two complete transfers of two times eight input data and two results can be executed.

c) (max 14 points)

Write VHDL-code for the data path described in a). In case you need control signals, you can for now simply assume that they are generated outside your current design. You should add comments to your code explaining what they do, however.

Describe all additional assumptions you make while writing the VHDL-code. In case you do not manage to complete all parts you should explain what is missing and describe it as "future work". This will not give full score, but still add to your points.

d) (max 12 points)

Write VHDL-code for a circuit that allows your adder to follow the valid-ready protocol.

Describe all additional assumptions you make while writing the VHDL-code. In case you do not manage to complete all parts you should explain what is missing and describe it as "future work". This will not give full score, but still add to your points.

Fill in your answer here

Format

a)
One alternative is to collect the eight numbers needed from two following bus cycles, and then use a adder tree with seven adders to perform the additions. An alternative that uses fewer adders, is to add two and two of the values arriving in one bus cycle, and then reuse the adders during the next bus cycle. At the same time the results from the two bus cycles are added together. A possible micro architecture for this solution is shown below.

b)
The solution here is dependent on the chosen solution in a). In any case, we need to make sure that we keep a result from the adder until the receiving entity is ready to read it. Hence, we must deactivate *data_in_ready* to tell the sending entity to wait. We also need to signal to the receiving entity when we have a valid result using *data_out_valid*. The sketch below is an example of how it could look for the micro architecture in this solution example.

c)
The VHDL code reflecting the micro architecture in a) is shown further below. Here, the function resize is used to increase the bit width stepwise. Such details are not necessary to achieve a full score.

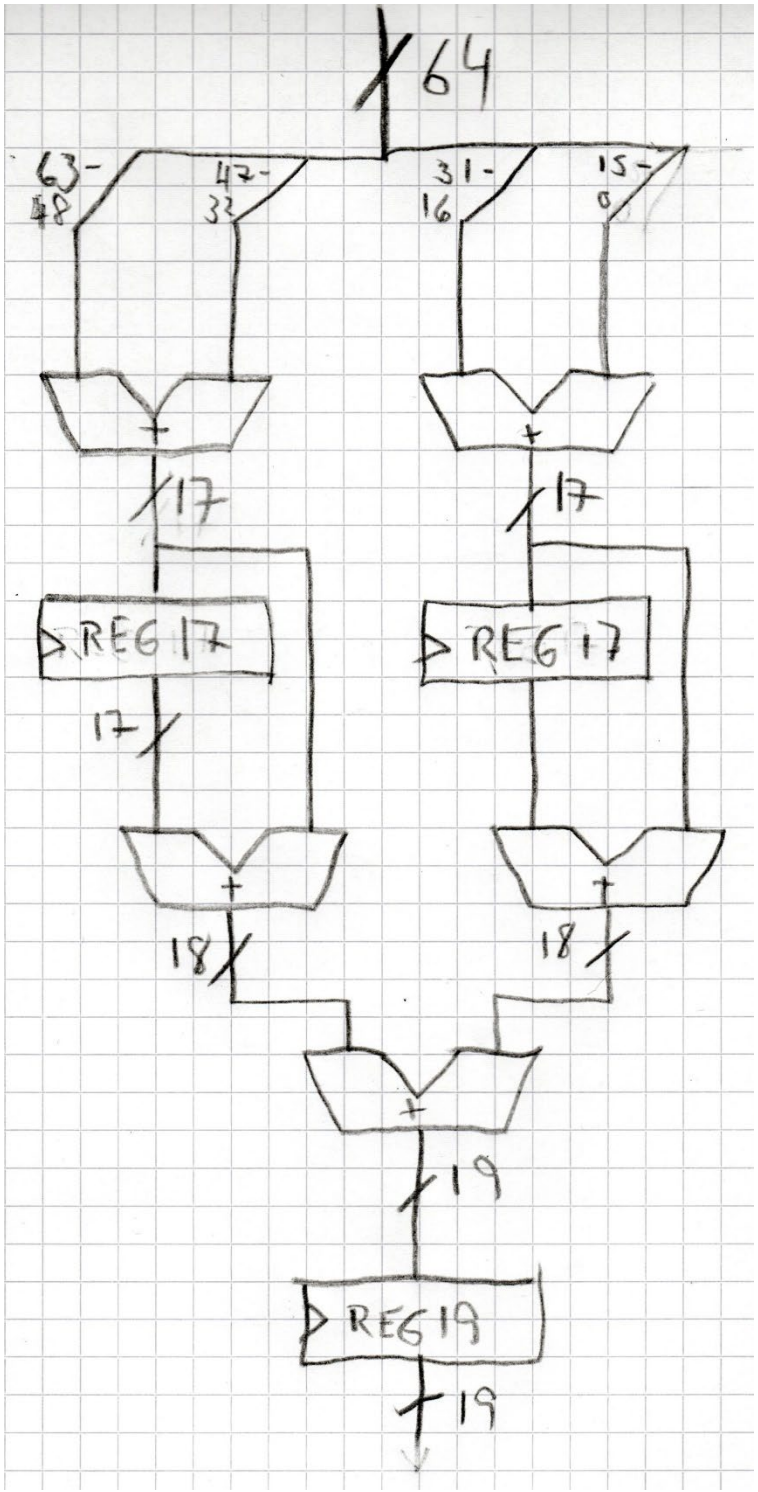
The critical path of the circuit is long, going through three adders, from the primary inputs to the output register. If a higher clock frequency would be needed, pipeline registers could be inserted, e.g., after *nextoutbusMSB* and *nextoutputLSB*. The solution also requires the inputs to be valid the whole clock period before the *store* signal is used to store (partial) results in registers. This is similar to the *data_in_valid* behavior.

Words: 0

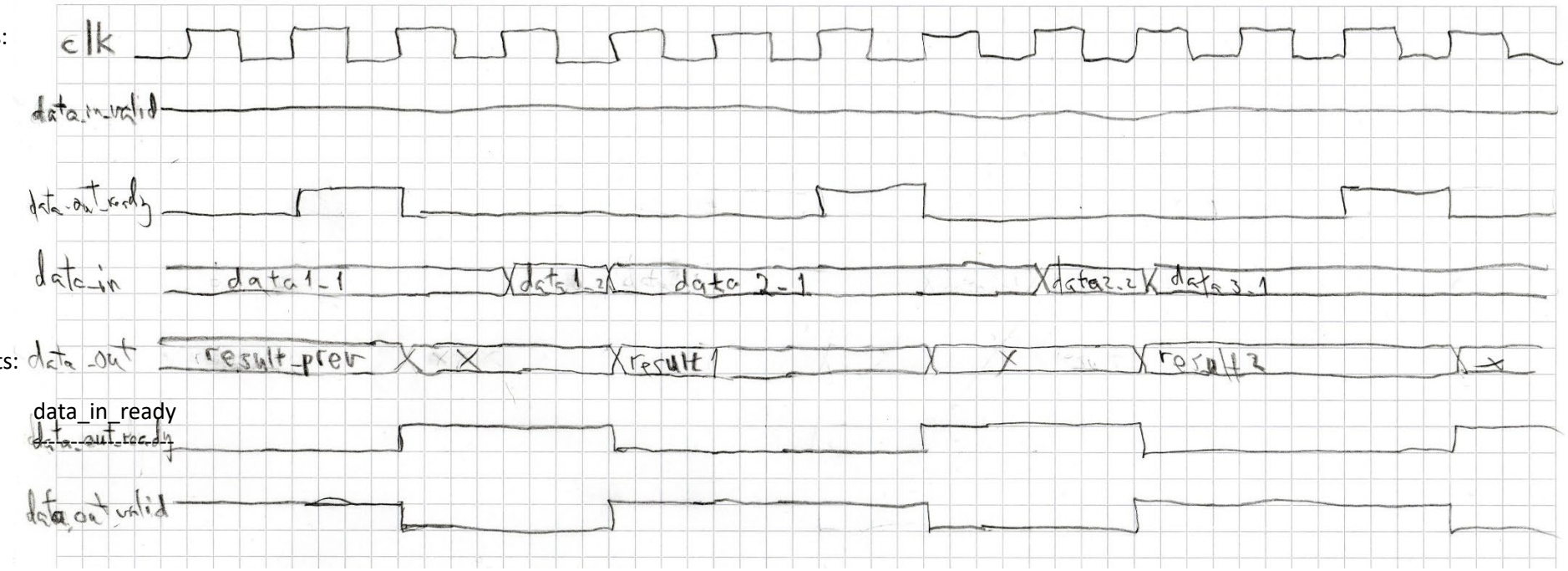
d)
A state machine is used to implement the protocol. No checking is performed to ensure that the input data is in sync (that we actually add to following sets of four numbers as wanted by the sender). Additional protocol rules could be added to ensure this. E.g., that the sender entity always deactivates *data_in_valid* in front of a new set of two data. This would be future work, however.

Maximum marks: 44

a)

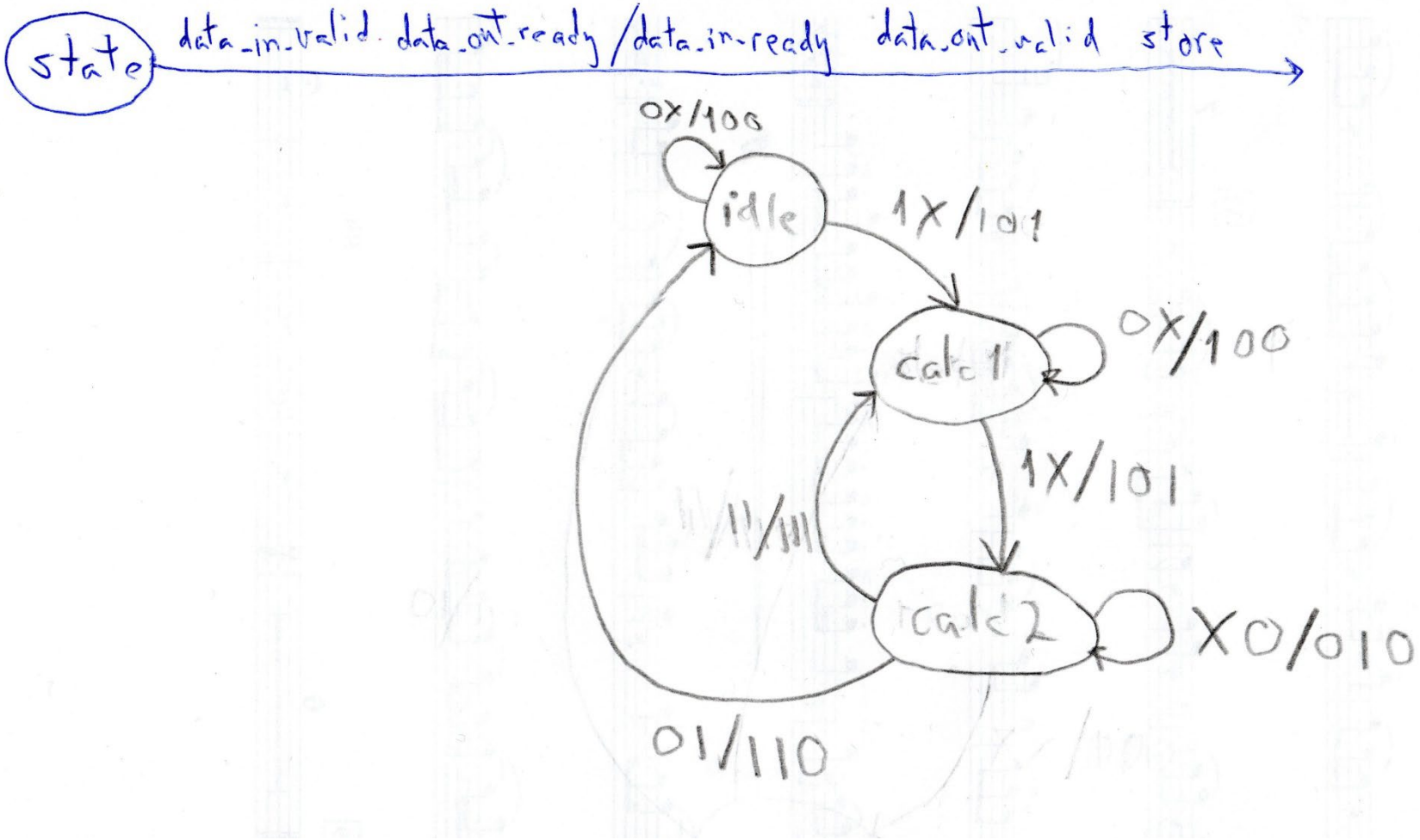


(g


adder inputs:
clk
data_in_valid
data_out_ready
data_in
data1_1
data1_2
data 2_1
data2_2
data 3_1
adder outputs:
data_out
result_prev
result1
result 2
data_in_ready
data_out_ready
data_out_valid

c)

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity Adder8DataPath is
    port (  clk, reset : in std_ulogic;
            store : in std_ulogic;  -- External control signal indicating new valid data on inbus
            inbus : in signed(63 downto 0);
            outbus : out signed(18 downto 0));
end Adder8DataPath;

architecture rtl of Adder8DataPath is
    signal inbus63_48: signed(15 downto 0);
    signal inbus47_32: signed(15 downto 0);
    signal inbus31_16: signed(15 downto 0);
    signal inbus15_0: signed(15 downto 0);
    signal nextpartsumMSB: signed(16 downto 0);
    signal nextpartsumLSB: signed(16 downto 0);
    signal partsumMSB: signed(16 downto 0);
    signal partsumLSB: signed(16 downto 0);
    signal nextoutbusMSB : signed(17 downto 0);
    signal nextoutbusLSB : signed(17 downto 0);
    signal nextoutbus : signed(18 downto 0);
    signal store_outbus : std_ulogic;  -- Delayed version of store_partsum for use in pipeline
begin

    process(inbus) -- Splitting of input bus for readability
    begin
        inbus63_48 <= inbus(63 downto 48);
        inbus47_32 <= inbus(47 downto 32);
        inbus31_16 <= inbus(31 downto 16);
        inbus15_0 <= inbus(15 downto 0);
    end process;

    process(inbus63_48, inbus47_32, inbus31_16, inbus15_0) -- Adding two and two numbers on inbus
    begin
        nextpartsumMSB <=  resize(inbus63_48, inbus63_48'length+1) + resize(inbus47_32, inbus47_32'length+1);
        nextpartsumLSB <=  resize(inbus31_16, inbus31_16'length+1) + resize(inbus15_0, inbus15_0'length+1);
    end process;

    process(clk, reset) -- Saving the first partial sums
    begin
        if (reset = '1') then
            partsumMSB <= (others => '0');
            partsumLSB <= (others => '0');
            store_outbus <= '0';
        elsif(clk'event and clk='1') then
            store_outbus <= store;
            if (store = '1') then
                partsumMSB <= nextpartsumMSB;
                partsumLSB <= nextpartsumLSB;
            end if;
        end if;
    end process;

    process(nextpartsumMSB, nextpartsumLSB, partsumMSB, partsumLSB, nextoutbusMSB, nextoutbusLSB)
    -- Adding the previous and the current two and two values on inbus and adding these results again
    begin
        nextoutbusMSB <= resize(partsumMSB, partsumMSB'length+1) + resize(nextpartsumMSB, nextpartsumMSB'length+1);
        nextoutbusLSB <= resize(partsumLSB, partsumLSB'length+1) + resize(nextpartsumLSB, nextpartsumLSB'length+1);
        nextoutbus <= resize(nextoutbusMSB, nextoutbusMSB'length+1) + resize(nextoutbusLSB, nextoutbusLSB'length+1);
    end process;

    process(clk, reset) -- Saving the final result to the outbus
    begin
        if (reset = '1') then
            outbus <= (others => '0');
        elsif(clk'event and clk='1') then
            if (store_outbus = '1') then
                outbus <= nextoutbus;
            end if;
        end if;
    end process;

end rtl;
```


state
data_in_valid. data_out_ready /data_in_ready data_out_valid store
0X/100
idle
1X/101
calc1
0X/100
1X/101
11/111
calc2
X0/010
01/110

```vhdl
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity StateMachine is
    port ( clk, reset, data_in_valid, data_out_ready : in std_ulogic;
           data_in_ready, data_out_valid, store : out std_ulogic );
end StateMachine;

architecture Behavioral of StateMachine is
    type state is (idle, calc1, calc2);
    signal curr_state, next_state : state;
begin

   comb_proc: process (curr_state, data_in_valid, data_out_ready) is
   begin
        case (curr_state) is
            when idle =>
                data_in_ready <= '1';
                data_out_valid <= '0';
                if data_in_valid = '1' then
                    next_state <= calc1;
                    store <= '1';
                else
                    next_state <= idle;
                    store <= '0';
                end if;
            when calc1 =>
                data_in_ready <= '1';
                data_out_valid <= '0';
                if data_in_valid = '1' then
                    next_state <= calc2;
                    store <= '1';
                else
                    next_state <= calc1;
                    store <= '0';
                end if;
            when calc2 =>
                data_out_valid <= '1';
                if (data_in_valid = '1') and (data_out_ready = '1') then
                    next_state <= calc1;
                    data_in_ready <= '1';
                    store <= '1';
                elsif (data_in_valid = '0') and (data_out_ready = '1') then
                    next_state <= idle;
                    data_in_ready <= '1';
                    store <= '0';
                else
                    next_state <= calc2;
                    data_in_ready <= '0';
                    store <= '0';
                end if;
            when others =>
                data_in_ready <= '0';
                data_out_valid <= '0';
                store <= '0';
                next_state <= idle;
            end case;
   end process;

   Synch_proc: process (clk, reset) is
   begin
        if reset = '1' then
            curr_state <= idle;
        elsif (clk'event and clk = '1') then
            curr_state <= next_state;
        end if;
   end process;

end Behavioral;
```